



# Parallélisme à base de thread

(PBT 2025-2026)

## TD2

Programmation par tâches  
OpenMP

teodora.hovi@cea.fr

marc.perache@cea.fr

Les objectifs de ce TD sont :

- Mise en pratique du modèle de programmation par tâches,
- Manipulation avec OpenMP.

## I Calcul de $\pi$ par Monte Carlo

### 1 Définition de la méthode

Le calcul de  $\pi$  est un problème fréquemment étudié pour l'apprentissage du parallélisme. L'une des méthodes les plus courantes est la méthode approchée de type *Monte Carlo*.

On construit un carré de côté  $c = 1$  et de centre  $O$ . En ce même centre, nous traçons un cercle de rayon  $r = 1$ . Pour comprendre la procédure d'approximation de  $\pi$ , nous avons besoin de connaître l'aire de ce cercle via la formule  $A = \pi r^2$ . Dans notre cas, nous obtenons  $A = \pi$ .

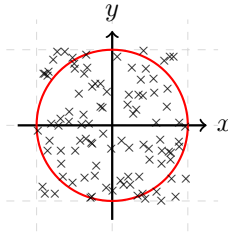


FIGURE 1 – Distribution aléatoire de points dans un carré unitaire.

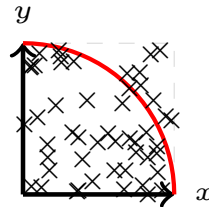


FIGURE 2 – Quart de cercle servant de cible pour le calcul de  $\pi$

FIGURE 3 – Calcul d'approximation de  $\pi$  par la méthode de Monte Carlo

La méthode de Monte Carlo consiste à lancer un nombre infini de fléchettes manière aléatoire en direction de la cible (cf Figure 2). Le nombre de fléchettes  $P$  se trouvant dans le cercle tend donc vers l'aire du quart de cercle, soit  $\pi/4$ .

En pratique,  $P$  se calcule également de la manière suivante :

$$P = \frac{\text{nb fléchettes dans le cercle}}{\text{nb total de fléchettes}}.$$

On se retrouve donc avec l'approximation de  $\pi$  telle que :

$$\pi \simeq 4 \times \frac{\text{nb fléchettes dans le cercle}}{\text{nb total de fléchettes}}.$$

Pour avoir l'approximation de  $\pi$ , on compte alors le nombre de fléchettes se trouvant dans le quart de cercle de la cible. Un point  $a$  de coordonnées  $(x, y)$  se trouve à l'intérieur du quart de cercle si la distance qui le sépare de l'origine  $O(0,0)$  est inférieure ou égale à 1, soit  $x^2 + y^2 \leq 1$  (cf. théorème de Pythagore).

## 2 Exercice

**Q.1:** A l'aide de la description du problème dans la section précédente, du programme séquentiel présent dans le fichier `Pi_MT/pi_seq.c`, écrire un programme parallèle à base de tâches OpenMP 3.0 où chaque thread lance un nombre  $N\_TASKS\_PER\_THREAD$  de tâches exécutant chacune  $N\_TRIALS\_PER\_TASK$ .

NB : On peut aussi imaginer que  $N\_TRIALS\_PER\_TASK$  est un nombre aléatoire servant à créer du déséquilibre entre thread.

## II Sparse Matrix Vector kernel (SpMV)

A l'instar des matrices denses, une matrice creuse est une matrice contenant un nombre important de valeurs nulles. Pour limiter l'empreinte mémoire liée au stockage de la matrice, seules les valeurs non nulles sont stockées. Différents formats existent pour arriver à ce résultat.

Dans le cadre de cet exercice, nous nous focalisons sur le format *Compressed Sparse Row* (CSR). Considérons une matrice creuse de taille  $N \times N$ , elle est alors stockée sous forme de trois vecteurs :

- `values` : stocke l'ensemble des valeurs non nulles ligne par ligne de la matrice, de taille `NNZ`.
- `JA`, un tableau d'entiers qui enregistre l'indice de colonne de chaque valeur, il est de taille `NNZ`.
- `IA`, un tableau d'entiers qui contient les pointeurs vers le début de chaque ligne dans les tableaux `values` et `JA`. La taille de ce vecteur est de  $N + 1$  éléments.

La signature du format *CSR* est donnée dans la portion de code suivante :

```

1  typedef struct
2  {
3      int m_nrows, m_nnz ;
4      double* m_values ;
5      int* m_ja ;
6      int* m_ia ;
7  } CSRMatrix_t;

```

Le code présent dans le répertoire `SpMV` contient les fonctions d'allocation et d'initialisation d'une matrice creuse au format `CSR`. Complétez les fichiers correspondants pour répondre aux questions.

**Q.2:** Le fichier `CSRMatrix.c` contient le squelette de la fonction `mult_CSR(CSR_Matrix_t*, double const*, double)` qui effectue le produit entre une matrice creuse et un vecteur. Remplissez cette fonction avec l'opération séquentielle correspondante.

**Q.3:** Proposez une parallélisation par tâches de l'algorithme précédent en utilisant la norme OpenMP 4.0 et en faisant en sorte que chaque tâche effectue un SpMV sur un sous-ensemble de lignes de la matrice.

**Q.4:** Quels sont les avantages apportent la parallélisation par tâches d'un tel algorithme ?

Ce TP est à rendre. Le code source ainsi qu'un rapport répondant aux questions et **détaillant l'implémentation** est à envoyer sur le lien suivant avant le 05/03/2026 à 23h59.

<sup>1</sup> <https://my.pcloud.com/#page=puplink&code=I2b7ZqF4bU1zxaLLolPccF77KDVmkMCVV>